

Establishing a Modern Application Security Program

The OWASP Top Ten lists are awareness documents, meant to bring awareness to the most critical risks of whichever topic they cover. They are not meant to be a complete list, only a starting place. In previous versions of this list we have prescribed starting an application security program as the best way to avoid these risks, and more. In this section we will cover how to start and build a modern application security program.

If you already have an application security program, consider performing a maturity assessment on it using [OWASP SAMM \(Software Assurance Maturity Model\)](#) or DSOMM (DevSecOps Maturity Model). These maturity models are comprehensive and exhaustive and can be used to help you figure out where you should best focus your efforts for expanding and maturing your program. Please note: you do not need to do everything in OWASP SAMM or DSOMM to be doing a good job, they are meant to guide you and offer many options. They are not meant to offer unattainable standards or describe unaffordable programs. They are expansive in order to offer you many ideas and options.

If you are starting a program from scratch, or you find OWASP SAMM or DSOMM 'too much' for your team right now, please review the following advice.

1. Establish a Risk Based Portfolio Approach:

- Identify the protection needs of your application portfolio from a business perspective. This should be driven in part by privacy laws and other regulations relevant to the data asset being protected.
- Establish a [common risk rating model](#) with a consistent set of likelihood and impact factors reflective of your organization's tolerance for risk.
- Accordingly measure and prioritize all your applications and APIs. Add the results to your [Configuration Management Database \(CMDB\)](#).
- Establish assurance guidelines to properly define coverage and level of rigor required.

2. Enable with a Strong Foundation:

- Establish a set of focused policies and standards that provide an application security baseline for all development teams to adhere to.

- Define a common set of reusable security controls that complement these policies and standards and provide design and development guidance on their use.
- Establish an application security training curriculum that is required and targeted to different development roles and topics.

3. Integrate Security into Existing Processes:

- Define and integrate secure implementation and verification activities into existing development and operational processes.
- Activities include threat modeling, secure design and design review, secure coding and code review, penetration testing, and remediation.
- Provide subject matter experts and support services for development and project teams to be successful.
- Review your current system development life cycle and all software security activities, tooling, policies, and processes, then document them.
- For new software, add one or more security activities to each phase of the system development life cycle (SDLC). Below we offer many suggestions of what you can do below. Ensure you perform these new activities on every new project or software initiative, this way you will know each new piece of software will be delivered at an acceptable security posture for your organizations.
- Select your activities to ensure your final product meets an acceptable level of risk for your organization.
- For existing software (sometimes called legacy) you will want to have a formal maintenance plan, please look below for ideas of how to maintain secure applications in the section called 'Operations and Change Management'.

4. Application Security Education:

- Consider starting a security champion program, or general security education program for your developers (sometimes called an advocacy or security awareness program), to teach them everything you wish they would know. This will keep them up to date, help them know how to do their work securely, and make the security culture where you work more positive. It often also improves trust between the teams and makes for a happier working relationship. OWASP supports you in this with the [OWASP Security Champions Guide](#), which is being expanded step by step.
- The OWASP Education Project provides training materials to help educate developers on web application security. For hands-on learning about vulnerabilities, try the [OWASP Juice Shop Project](#), or [OWASP WebGoat](#). To stay current, come to an [OWASP AppSec Conference](#), [OWASP Conference Training](#), or local [OWASP Chapter](#) meetings.

5. Provide Management Visibility:

- Manage with metrics. Drive improvement and funding decisions based on the metrics and analysis data captured. Metrics include adherence to security practices and activities, vulnerabilities introduced, vulnerabilities mitigated, application coverage, defect density by type and instance counts, etc.
- Analyze data from the implementation and verification activities to look for root cause and vulnerability patterns to drive strategic and systemic improvements across the enterprise. Learn from mistakes and offer positive incentives to promote improvements.

Establish & Use Repeatable Security Processes and Standard Security Controls

Requirements and Resource Management Phase:

- Collect and negotiate the business requirements for an application with the business, including the protection requirements with regard to confidentiality, authenticity, integrity and availability of all data assets, and the expected business logic.
- Compile the technical requirements including functional and nonfunctional security requirements. OWASP recommends you use the [OWASP Application Security Verification Standard \(ASVS\)](#) as a guide for setting the security requirements for your application(s).
- Plan and negotiate the budget that covers all aspects of design, build, testing and operation, including security activities.
- Add security activities to your project schedule.
- Introduce yourself as the security representative at the project kick off, so they know who to talk to.

Request for Proposals (RFP) and Contracting:

- Negotiate the requirements with internal or external developers, including guidelines and security requirements with respect to your security program, e.g. SDLC, best practices.
- Rate the fulfillment of all technical requirements, including a planning and design phase.
- Negotiate all technical requirements, including design, security, and service level agreements (SLA).
- Adopt templates and checklists, such as [OWASP Secure Software Contract Annex](#).
Note: *The annex is for US contract law, so please consult qualified legal advice before using the sample annex.*

Planning and Design Phase:

- Negotiate planning and design with the developers and internal shareholders, e.g. security specialists.
- Define the security architecture, controls, countermeasures and design reviews appropriate to the protection needs and the expected threat level. This should be supported by security specialists.
- Rather than retrofitting security into your applications and APIs, it is far more cost effective to design the security in from the start. OWASP recommends the [OWASP Cheat Sheets](#) and the [OWASP Proactive Controls](#) as a good starting point for guidance on how to design security included from the beginning.
- Perform threat modelling, see [OWASP Cheat Sheet: Threat Modeling](#).
- Teach your software architects secure design concepts and patterns and ask them to add them to their designs where possible.
- Examine data flows with your developers.
- Add security user stories alongside all of your other user stories.

Secure Development Lifecycle:

- To improve the process your organization follows when building applications and APIs, OWASP recommends the [OWASP Software Assurance Maturity Model \(SAMM\)](#). This model helps organizations formulate and implement a strategy for software security that is tailored to the specific risks facing their organization.
- Provide secure coding training to your software developers, and any other training you think will help them create more robust and secure applications.
- Code review, see [OWASP Cheat Sheet: Secure Code Review](#).
- Give your developers security tools, then teach them how to use them, especially static analysis, software composition analysis, secret, and [Infrastructure-as-Code \(IaC\)](#) scanners.
- Create guardrails for your developers, if possible (technical safeguards to steer them towards more secure choices).
- Building strong and usable security controls is difficult. Offer secure defaults whenever possible, and create 'paved roads' (making the easiest way also the most secure way to do something, the obvious preferred way) whenever possible. The [OWASP Cheat Sheets](#) are a good starting point for developers, and many modern frameworks now come with standard and effective security controls for authorization, validation, CSRF prevention, etc.
- Give your developers security-related IDE plugins and encourage them to use them.
- Provide them a secret management tool, licenses, and documentation on how to use it.

- Provide them a private AI to use, ideally set up with a RAG server full of useful security documentation, prompts your team has written for better outcomes, and an MCP server that calls the security tooling of choice for your org. Teach them how to use AI safely, because they are going to do it whether you like it or not.

Establish Continuous Application Security Testing:

- Test the technical functions and integration with the IT architecture and coordinate business tests.
- Create “use” and “abuse” test cases from technical and business perspectives.
- Manage security tests according to internal processes, the protection needs, and the assumed threat level by the application.
- Provide security testing tools (fuzzers, DAST, etc.), a safe place to test, and training on how to use them, OR do the testing for them OR hire a tester
- If you require a high level of assurance, consider a formal penetration test, as well as stress testing and performance testing.
- Work with your developers to help them decide what they need to fix from the bug reports, and ensure their managers give them time to do it.

Rollout:

- Put the application in operation and migrate from previously used applications if needed.
- Finalize all documentation, including the change management database (CMDB) and security architecture.

Operations and Change Management:

- Operations must include guidelines for the security management of the application (e.g. patch management).
- Raise the security awareness of users and manage conflicts about usability vs. security.
- Plan and manage changes, e.g. migrate to new versions of the application or other components like OS, middleware, and libraries.
- Ensure all apps are in your inventory, with all important details documented. Update all documentation, including in the CMDB and the security architecture, controls, and countermeasures, including any runbooks or project documentation.
- Perform logging, monitoring, and alerting for all apps. Add it if it's missing.
- Create processes for effective and efficient updating and patching.

- Create regular scanning schedules (hopefully dynamic, static, secrets, IaC, and software composition analysis).
- SLAs for fixing security bugs.
- Provide a way for employees (and ideally also your customers) to report bugs.
- Establish a trained incident response team that understands what software attacks look like, observability tooling.
- Run blocking or shielding tools to stop automated attacks.
- Annual (or more often) hardening of configurations.
- At least annual penetration testing (depending upon the level assurance required for your app).
- Establish processes and tooling for hardening and protecting your software supply chain.
- Establish and update business continuity and disaster recovery planning that includes your most important applications and the tools you use to maintain them.

Retiring Systems:

- Any required data should be archived. All other data should be securely wiped.
- Securely retire the application, including deleting unused accounts and roles and permissions.
- Set your application's state to retired in the CMDB.

Using the OWASP Top 10 as a standard

The OWASP Top 10 is primarily an awareness document. However, this has not stopped organizations from using it as a de facto industry AppSec standard since its inception in 2003. If you want to use the OWASP Top 10 as a coding or testing standard, know that it is the bare minimum and just a starting point.

One of the difficulties of using the OWASP Top 10 as a standard is that we document AppSec risks, and not necessarily easily testable issues. For example, [A06:2025-Insecure Design](#) is beyond the scope of most forms of testing. Another example is testing whether in-place, in-use, and effective logging and monitoring are implemented, which can only be done with interviews and requesting a sampling of effective incident responses. A static code analysis tool can look for the absence of logging, but it might be impossible to determine if business logic or access control is logging critical security breaches. Penetration testers may only be able to determine that they have invoked incident response in a test environment, which is rarely monitored in the same way as production.

Here are our recommendations for when it is appropriate to use the OWASP Top 10:

Use Case	OWASP Top 10 2025	OWASP Application Security Verification Standard
Awareness	Yes	
Training	Entry level	Comprehensive
Design and architecture	Occasionally	Yes
Coding standard	Bare minimum	Yes
Secure Code review	Bare minimum	Yes
Peer review checklist	Bare minimum	Yes
Unit testing	Occasionally	Yes
Integration testing	Occasionally	Yes
Penetration testing	Bare minimum	Yes
Tool support	Bare minimum	Yes
Secure Supply Chain	Occasionally	Yes

We would encourage anyone wanting to adopt an application security standard to use the [OWASP Application Security Verification Standard](#) (ASVS), as it's designed to be verifiable and tested, and can be used in all parts of a secure development lifecycle.

The ASVS is the only acceptable choice for tool vendors. Tools cannot comprehensively detect, test, or protect against the OWASP Top 10 due to the nature of several of the OWASP Top 10 risks, with reference to [A06:2025-Insecure Design](#). OWASP discourages any claims of full coverage of the OWASP Top 10, because it's simply untrue.